Kazy Noor Sir | 7th week

26.7.26

Syntax Directed Definition

CFG: আগে থেকে context থাকে না

Ex- তিনটা variable a, b, c

এখানে যে variable ব্যবহার আগে থেকে কোনোকিছু define করা থাকে না

C lang. is not CFG

Semantic: নিয়মের ক্রম, associativity

Ex: char a; int b, c;

a = b + c ✗ semantic error

Semantic rules: কোন কোন obj কে associate করতে পারব তার rules আছে

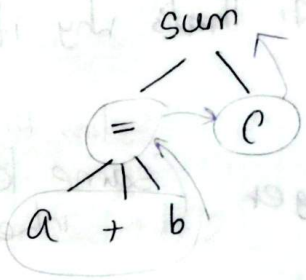
What is SDD?

⇒ SDD is a CFG, এখানে কিছু attribute থাকবে এবং attribute ব্যবহারের জন্য semantic rules থাকবে

⇒ Attribute 2 types — synthesized, inherited

Synthesized:

c = a + b



execution bottom to up, যখন leaf level থেকে value গুলো calculate হয়ে top level এ যাবে

Inherited

⇒ পাশাপাশি ও থাকতে পারে, leaf এও থাকতে পারে

⇒ একই level এর member সেগুলো সাধারণত value share করবে, তাদের নিচের level থেকেও value নিবে। That's why it is called inherited

Ex: Manager → Manager (same level inheritence) — class to class inheritance

Worker → Manager (নিচের level থেকে inheritence) — class to object inheritence

CEO → Manager (Top to bottom)

Difference betⁿ inherited and synthesized

L-attribute: Inherited attribute left to right এ

Translation schemes:

$a = b + c$,

b+c যোগ করে a তে রাখা valid কিনা এভাবে দেখবে

Synthesized att

Ex: $E \rightarrow E1 + E2$

⇒ E is determined by

⇒ $E_1 + E_2$

Inherited att (L-type)

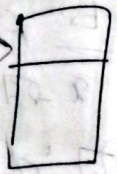
Ex: char x, y, z

x এর property টা 'char' determine করবে, left to right propagation হবে. x, y, z সবগুলোর property share

⇒ char *x

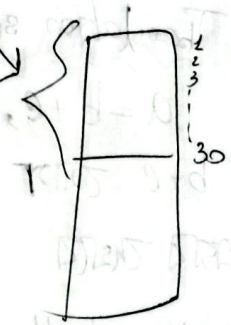
first element টা কি indicate করা আছে এখান থেকে বাকিগুলোর address determine করতে হবে

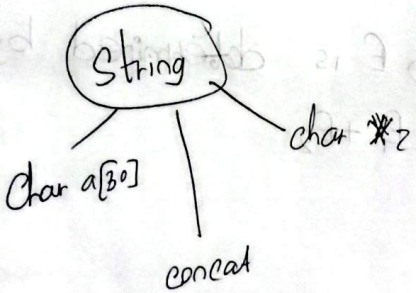
Infinite number



⇒ char x[30].

finite number





Parse tree: attribute গুলোকে একটা connectivity map এর মাধ্যমে diagram আকারে show করি

Symbol Parse tree	Annotated parse tree
1) Conceptual representation	1) machine level এ translate করার জন্য annotation দেওয়া থাকে

☆☆ Diff. betn symbol parse tree and annotated parse tree and why annotated parse tree is needed

a = 5 + 3 * 4

b c x

2) b = 5

c = 3



2) leaf এ lex value receive করে

digit. lexvalue = 5

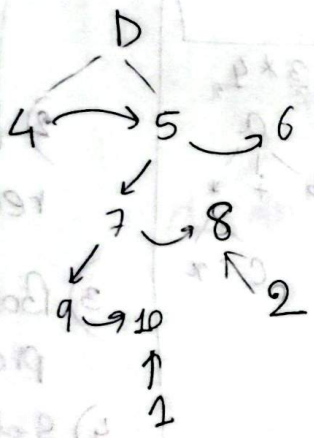
3) Bottom up process

4) S-att, L-att থাকা যায়

Dependency graph

⇒ Annotated parse tree তে যেখানে যেখানে attribute মূল্যের মধ্যে dependency থাকে, এই dependency টা বোঝা যায় grammar থেকে

⇒ Dependency graph এর ক্ষেত্রে annotated parse tree এর difference হচ্ছে dependency graph is a directed graph



☆☆Exam এ আসবে

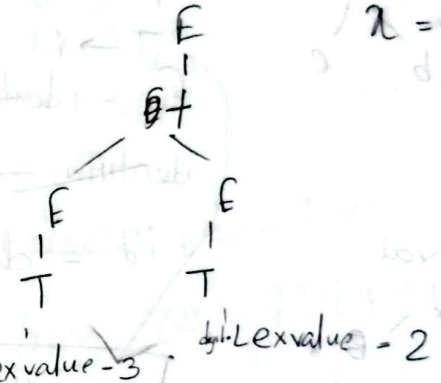
Ex: A CFG is given

E → E+E
T = id
T = num

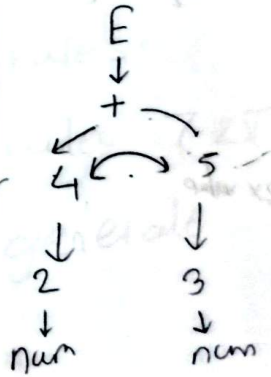
E → T

Expression:

x = y + 2
x = 3 + 2



Dependency graph



S-type dependency

4 → 5
similar domain
associativity depend on each other
L-type dependency
similar

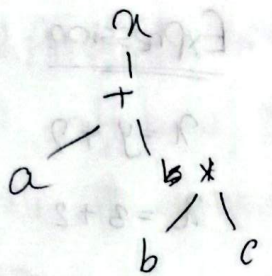
Kazy Noor Sir | Semantic Rules | 27.7.27

int x,a,b,c

$x = a + b * c$ → এটার জন্য CFG লিখলে

↓

$E = E \mid E_1 \text{ op } E_2 \mid (E_2)$



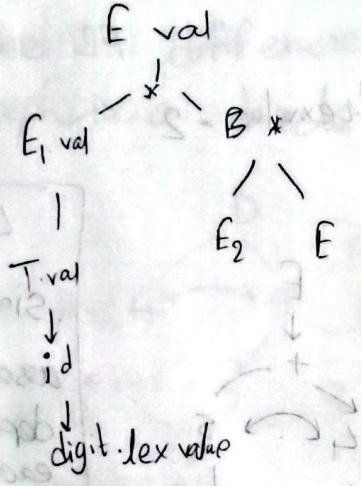
op = + / - / * / "/"

T → id

E = identifier id

identifier → id

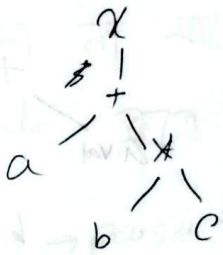
id = digit



L-attl	S-attl
T.type = id.type	T.val = id.val
Type নির্ণয়	Value নির্ণয়

Conceptual

Parse tree থেকে associativity, domain, type এগুলো বোঝা যায় না



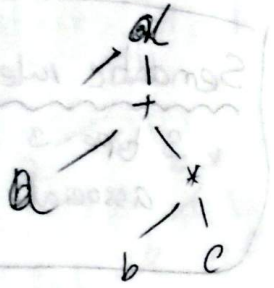
Semantic rules:
S type এ L type যুক্ত associate

যে generalised parse tree কে annotated parse tree তে রূপান্তর করতে হবে

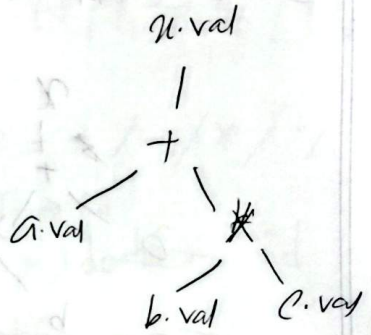
CFG তে product^n rules থাকে এগুলো semantic rules.

Product^n rules থেকে semantic rules generate করতে হবে

Parse tree



Annotated parse tree



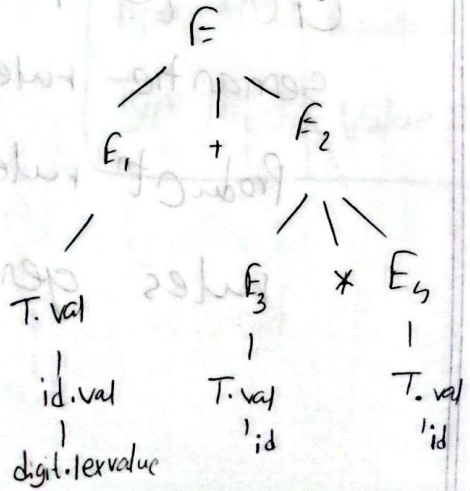
যাদের একটা CFG আছে

$E = E_1 \mid (E_1) \mid E_1 + E_2 \mid E_3 * E_4$

$E = T$

$T \rightarrow id$

$id \rightarrow digit.lexvalue$



Quest^n: express^n given. draw parse tree and annotated parse tree যদি quest^n এ কোথাও না থাকে CFG, তাহলে তোমাকে generate করে নিতে হবে

CFG → parse tree → ~~semantic~~ annotated parse tree

↓

semantic rule

Kazy Nur Sir | 8th week

9.8.26

S-attribute limitat^n

bottom up হতে হলে তার same domain এ

L-attribute

⇒ different class, different attri

student class এ student attribute

⇒ যেই synthesized attri, আরেক inherited attri diff

⇒ Communicat^n ও abstract^n / encapsulat^n property আছে (parallel communicat^n)

Exam

☆ Synthesized attri are inherited type,

☆ synthesized attri can work parallel or bottom up - Justify ~~bottom~~ it

Comparison ← with example.

Dependency graph | parse tree তে যেভাবে direct যেতে

⇒ direct আছে

⇒ direct নাকি to show their inter dependencies

Exam

☆ Express^n যেটার parse tree and dependency graph

add (identry, L.in)

⇒ এই দুইটাকে associate করা হচ্ছে Funct^n টার সাথে

5+3*4

1) in → 5
2) in → 3
3) in → 4

4) এর → L → id → d →
5)

float a, b, c

a → L. id
b → L. id
c → L. id

Final call

Exam

⇒ Why dependency graph is required

Ans: Code generat" Communicat" ক্ষেত্র translate করতে

⇒ diff bet dependency graph and parse tree.

given in slide

Exam

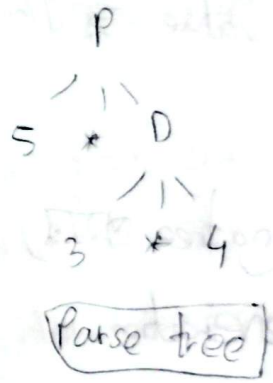
⇒ Ambiguous grammar এর ক্ষেত্রে cyclic dependency graph related or not

Evaluat": যে rules/grammar ব্যবহার করেই যেকোন rules টার expressn এর meaning কতটুক বের করে নিয়ে আসবে

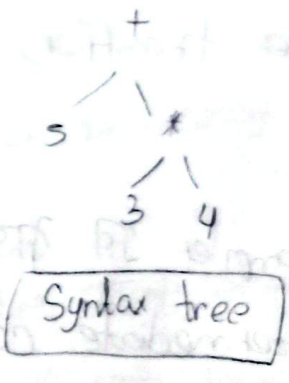
CT-2 next week [slide দেখা]

Syntax related compilat" দেখাবে

Syntax tree: unnecesary symbols of pase tree বাদ দিয়ে একটা simplified form



Parse tree



Syntax tree

node বর্জিত → এভাবে

কোনো operator নাই

Exam Why syntax tree is important

Ques ১ memory mapping করতে কী কী লাগে syntax tree

Ans: To specify the order with direction
Memory mapping

⇒ mkleaf(id, entry) → কোনো একটা value entry দিবে

⇒ mkleaf(num, val) → variable এর মধ্যে number transffer, given value থাকবে

Compiler এর কাজ parsing tree পর্যন্ত, intermediate code generation

Kazy Noor Sir

16.8.26

Issues in Design of code generator

A simple target machine model দিয়ে imp. এছাড়া কী কী প্রশ্ন আসবে

এরপরের page গুলো imp না,

Then flow graph based on basic blocks

⇒ Basic block এর মধ্যে inter dependency থাকার importance কী?

Ans: inter dependency violate হলে semantic error

Error debug করার জন্য একটা detailed diagram দেখাতে হবে

⇒ Digram ১ Label B_1, B_2 দেওয়া mandatory

1) a = b + c
2) d = b + c
3) e = a * d
4) f = b + c
5) g = e - f

1 → b + c

1 → a, d, f

2. e = a * d = 1 * 1

= *(1, 1)

it means loop হচ্ছে

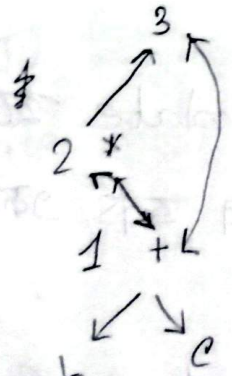
3. g = e - f

= 2 - 1

= -(2, 1)

Bidirectional

DAG



Intermediate code generation

Three address code

⇒ তিনটা operand
+ operator

Quadruple

⇒ তিনটা operands থাকবে
সাথে একটা result রাখার জন্য
+ operator

Triples

⇒ operand রাখা হবে
না। একটা positional address
রাখা হয়।

⇒ $\overset{1}{c} = \overset{2}{a} - (\overset{3}{b} + \overset{4}{c}) + \overset{5}{g} * \overset{6}{f}$

= sign এর RHS এ

1) Operand 5 টা

2) Operator 5 টা

This is not in three all code

We need to convert it

⇒ Three address code এর rule অনুযায়ী = operand এর RHS এ max দুইটা operand থাকতে পারে

$c = a - (b+c) + g * f$

$t_1 = a$
$t_2 = b+c$
$t_3 = t_1 - t_2$

Transform করার পদ্ধতি

TAC হয় যা, operand একটা

So we write

$t_1 = b+c$

$t_2 = a-(b+c)$ → $t_3 = g * f$

$c = t_2 + t_3$

TAC

index	operator	A_1	A_2	Result
1	+	b	c	t_1
2	−	a	t_1	t_2
3	*	g	f	t_3
4	+	t_2	t_3	c

Triples

indexing 0 থেকে start

index	op	A_1	A_2	Result
0	+	b	c	t_1
1	−	a	t_2 0	t_2

directly operand use না করে প্রথম indexing টা তে বসিয়ে করব

2	*	g	f	t_3
3	+	1	2	e

Importance.

1) TAC only for error debug
2) Triple for index mapping
3) Quadrapole " code implementation

Kazy Nook

23.8.26

Ques
for any intermediate code generat", triples, quadroples, TAC why necessary

প্রথম slide এবং 2nd slide final এ 2 টা question

Ques
⇒ intermediate code geration বলতে কী বোঝায়
এর function কী
Design issues of intermediate code generat"
Register keyword as example and give discuss"

Ex: Load ...

scenario based Flow graph যেটা code generat"

{CT-2 on ~~first~~ second slide} Bring pencil

slid-3 থেকে খুব বেশি, design issues, register allocat" example, flow graph, DAG, why acyclic, value no., representat"

Run-Time Environment → এখানে 2 ধরনের allocat"

⇒ How do we allocate the space for the generated target code and the data object of our source program?

Ans: Compilat" এর সময় কিছু কিছু data allocate করা করা, int a . 16 bit পরিমাণ space allocate করা variable এর জন্য. compilat" এর সময় এই allocat" টা হয়ে যায়, পুরা running process এই 'a' এর space same থাকে. This is static allocation (Compilation → static allocat")

চাইলে deallocate করতে পারবো running time এ

Running time এ যে data allocat" করা হয় সেটা dynamic memory allocation.

Run time support package is loaded together with the generated target code
(.exe file টা set করে চলে আসে এমন)

The structure of the run-time support packag

depends on the semantics of the programming language

(Target program এর সাথে Semantics টা সংগতিপূর্ণ কিনা)

⇒ Runtime environment 4 type

Procedure activat^n

⇒ প্রত্যেকটা procedure এর execut^n → activat^n

⇒ Procedure এর load time, first step, last step কতক্ষণ পর্যন্ত চলবে এটা তার lifetime

⇒ Static recurss^n → lifetime এ add হবে

⇒ user controlled entry তে কতক্ষণ procedure survive করবে

Quest^n

Exa: code segment টা তে user input নেওয়া কতবার recurss^n টা lifetime এ add হবে আর কতবার add হবে না

user থেকে input নিচ্ছে how many entries user gave 5 so loop হবে 5 বার

যদি loop টা আগে থেকে define করা থাকে তাহলে তার lifetime এর অংশ

(i)
```
int i, j;
for(i=0; i<=1000; i++)
  scanf("%d", &i);
    for
```

⇒ i, j আগে থেকে defined

(ii)
```
scanf("%d, %d", i, j);
    for(i=0; i<=1000; i)
      scanf
```

⇒ i, j input নিয়ে তার উপর loop dependent

⇒ For (ii) lifetime হবে process এর first থেকে last execution → user input এর উপর loop dependent

⇒ For (i) lifetime will be total execution time of procedure for individual 'i' and 'j'

Run time memory allocation code example

```
scanf (i,j)
int i, j;
  int a = 10, b = 30;
char *fa ( ) {
 char *i,c ;
  c = malloc (...);
       heap
```

pointer এর মাধ্যমে memory address কোন static

Ques

⇒ particular কোন code segment what are the portions for which it will be dynamic memory allocation

⇒ Give code and mention where is static memory allocation and which one is dynamic memory Allocation (explain in details)

⇒ scenario বের করে total code কী হবে পাবে, static or memory element কী হবে পাবে, symbol table এ থাকবে.

Activat" records | also called frames

Storage Allocation strategies

Restrict"

⇒ যখন একটা local funct" call হবে যার মাধ্যমে করা হবে সে তাকে reference করে অন্য জায়গায় বসাতে পারে ??

Heap allocat" → Imp

heap এর memory free করা যাবে program থেকে

free()

খুব বেশি দরকার না হলে user থেকে input নেওয়ার দরকার নেই

Activat" tree

diag representat" of a procedure when it entries and when it exists after execution

root → activat" of main program

node → " " procedure

Ques

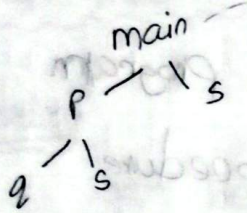
⇒ code দেওয়া থাকবে এটার activat" tree কী হবে?

procedure নিয়ে then draw করবে

Control stack

মানে হচ্ছে memory তে আছে (run time)

Procedure নিয়ে



Flow control

DFS

depth first

⇒ Why flow control is DFS?

Root থেকে leaf পর্যন্ত

একটা program full চেষ্টা করে (upto leaf) then পাশের টা হয়ে যায়

Variable scope

⇒ Main funct এর variable অন্য কোনো funct এ কাজ করবেনা caz it's local scope

(i) Application

(ii) Presentation

(iii) Sc

Runtime Storage Org

Static আর dynamic memory allocat^n এ

tread off আছে

যদি এই treadoff handling করতে না পারি error দিবে

One of the error is Danging reference

→ Returning address of local variable to funct^n call (given in slide example code)

Solution: manually create and deallocate.

Parameter Passing method

1) Call by value

Changing a variable value in funct^n doesn't affect the actual argument

depends on signal register : Hardware ই pressure বাড়ে

2) Call by reference

Allows modificat^n of actual argument

⇒ Costs more in terms of execut^n time

⇒ Hardware platform pressure

Address related value change হয় Register min. 2 টা লাগে

3) Call by name

SQR funct^n given in slide

funct^n ১ একবার 5 যাবে then 6 যাবে

SQR (i++)

Theory project | CT-3 Project marks

Project: Hardware platform target করে

DAG থেকে flowchart generate → P-1

What will be the engaged Registers → P-2

Implementat^n → Code, Presentat^n, Result → P-3

MUST include intermediate code generat^n

Parse tree if required

↓ to execut^n plan (এটাই DAG)